Gestión de la Información en la Web
Curso 2026-27
Práctica 7 – API REST

Fecha de entrega: domingo 15 de noviembre 2026, 23:55h

Entrega de la práctica
La entrega de la práctica se realizará a través del Campus Virtual de la asignatura mediante un fichero `pr7.py`. El esqueleto de este fichero se puede descargar del Campus Virtual.

Lenguaje de programación
Python 3.13 o superior.

Calificación
Se medirá la corrección mediante tests de unidad. Además de la corrección, se valorará la **calidad, concisión y claridad del código**, la incorporación de **comentarios** explicativos, su **eficiencia** tanto en tiempo como en memoria y la puntuación obtenida en Pylint.

Declaración de autoría e integridad
Todos los ficheros entregados contendrán una cabecera en la que se indique la asignatura, la práctica, el grupo y los autores. Esta cabecera también contendrá la siguiente declaración de integridad:

> *Declaramos que esta solución es fruto exclusivamente de nuestro trabajo personal. No hemos sido ayudados por ninguna otra persona o sistema automático ni hemos obtenido la solución de fuentes externas, y tampoco hemos compartido nuestra solución con otras personas de manera directa o indirecta. Declaramos además que no hemos realizado de manera deshonesta ninguna otra actividad que pueda mejorar nuestros resultados ni perjudicar los resultados de los demás.*

No se corregirá ningún fichero que no venga acompañado de dicha cabecera.

En esta práctica implementaremos una API REST muy pequeña que nos permitirá operar con asignaturas: añadirlas, consultarlas, editarlas y borrarlas. Este servicio debe implementarse en **Flask** y deberá escuchar en el **puerto 5000**.

Para realizar la práctica debéis descargar el esqueleto del servicio web que está en el Campus Virtual y usarlo como base. Este fichero incluye el código básico de un servidor Flask y únicamente en necesario definir e implementar las distintas rutas.

No es necesario que el servicio disponga de *persistencia*, es decir, podéis considerar que cada vez que se arranca éste comienza con un estado vacío sin ninguna asignatura. Si aún así queréis obtener persistencia podéis utilizar ficheros (de texto o binarios) o conectar directamente con una base de datos (SQLite por ejemplo). En la próxima práctica se tratará cómo obtener persistencia usando una traducción automática entre Python y MongoDB.

1. Esquema de las asignaturas

En nuestra API representaremos las asignaturas únicamente mediante JSON con los siguientes campos y tipos:

```
{
  "id": int,
  "nombre": str,
  "numero_alumnos": int,
  "horario: [
            {
              "dia": str,
              "hora_inicio": int,
              "hora_final": int
            }
          ]
}
```

Nótese que `horario` es una lista de cero, uno o varios documentos JSON, donde cada uno de estos documentos contendrá tres campos (`dia`, `hora_inicio` y `hora_final`). El campo `id` es incluido automáticamente por la API al recibir asignaturas, es decir, `id` **no debe aparecer en las asignaturas que se envían para añadir o reemplazar** ni puede modificarse mediante `PATCH`, pero el servidor sí que lo incluirá en las respuestas cuando el usuario consulte una determinada asignatura. Los campos de tipo `int` deben contener números enteros, por lo que valores como `20.5`, `"20"` o `true` **no son válidos**[1].

Por simplicidad, supondremos que los identificadores son números naturales incrementales comenzando en 0.

2. Rutas de la API REST

A continuación se explican las distintas rutas a las que deberá atender el API REST. Para cada una de ellas se incluyen varios **ejemplos** realizados desde el intérprete interactivo de Python con la biblioteca `requests` (la misma que usan los tests de unidad). Las líneas que comienzan por «`>>>`» son las invocaciones y el resto es la salida obtenida. Los ejemplos se ejecutan en el orden en que aparecen sobre un servidor recién arrancado. Para mostrar las respuestas de manera compacta, los ejemplos suponen que se ha ejecutado previamente el siguiente código, que define la función `muestra` para imprimir el código de estado y, si lo hay, el contenido JSON de la respuesta:

```
import json
import requests
```

[1] Cuidado: en Python `isinstance(True, int)` devuelve `True`, ya que `bool` es una subclase de `int`.

```
URL = "http://localhost:5000"

def muestra(r):
    print(r.status_code, r.reason)
    if r.content:
        print(json.dumps(r.json(), indent=2))
```

Para realizar estas mismas peticiones también podréis utilizar extensiones del navegador como «Yet Another REST Client» o «RESTED», e incluso aplicaciones con interfaz gráfica como «Insomnia» o «Postman».

Todas las respuestas que tengan contenido deben ser JSON, es decir, deben incluir la cabecera «`Content-Type: application/json`».

2.1. `/asignaturas`

Mediante esta ruta se podrá añadir una nueva asignatura, borrar todas las asignaturas y consultar las asignaturas:

- `DELETE`: Elimina todas las asignaturas que existan, devolviendo el código «**204 No Content**». Ejemplo:

```
>>> muestra(requests.delete(f"{URL}/asignaturas"))
204 NO CONTENT
```

- `POST`: Añade una nueva asignatura. Se debe incluir en la petición la asignatura a incluir representada como JSON sin ningún campo `id`. Si la petición **no tiene cuerpo**, la asignatura no cumple con el esquema o tiene tipos incorrectos en los campos, se devolverá el código «**400 Bad Request**»[2], en otro caso se devolverá el código «**201 Created**» y el contenido «`{"id": N}`» donde N es el identificador único asignado a la asignatura que se acaba de crear. Ejemplos

```
>>> muestra(requests.post(f"{URL}/asignaturas", json={
...     "nombre": "BD", "numero_alumnos": 70,
...     "horario": [{"dia": "lunes", "hora_inicio": 16, "hora_final": 18}]}))
201 CREATED
{
  "id": 0
}
>>> muestra(requests.post(f"{URL}/asignaturas", json={
...     "nombre": "GIW", "numero_alumnos": 40,
...     "horario": [{"dia": "martes", "hora_inicio": 14, "hora_final": 16}]}))
201 CREATED
{
  "id": 1
}
>>> muestra(requests.post(f"{URL}/asignaturas", json={
...     "nombre": "SGDI", "numero_alumnos": 25, "horario": []}))
201 CREATED
{
  "id": 2
}
>>> muestra(requests.post(f"{URL}/asignaturas", json={
```

[2] Cuidado: `request.get_json()` puede dar problemas con peticiones sin cuerpo JSON. Consultad su parámetro `silent`.

```
...    "nombre": "AW", "numero_alumnos": 55,
...    "horario": [{"dia": "jueves", "hora_inicio": 10, "hora_final": 12},
...                {"dia": "viernes", "hora_inicio": 10, "hora_final": 12}]}))
201 CREATED
{
  "id": 3
}
>>> muestra(requests.post(f"{URL}/asignaturas", json={"asignatura": "mal formada"}))
400 BAD REQUEST
>>> muestra(requests.post(f"{URL}/asignaturas"))
400 BAD REQUEST
```

- `GET`: permite obtener las URL de las asignaturas. Si no se pasa ningún parámetro, se devolverá un JSON `{"asignaturas": L}` donde L es una lista de URL para acceder a las distintas asignaturas **en el orden en que fueron añadidas**. Ejemplo:

```
>>> muestra(requests.get(f"{URL}/asignaturas"))
200 OK
{
  "asignaturas": [
    "/asignaturas/0",
    "/asignaturas/1",
    "/asignaturas/2",
    "/asignaturas/3"
  ]
}
```

  Esta ruta también admite los parámetros `page` y `per_page` (si aparece uno siempre debe aparecer el otro) para indicar el número de página solicitada (la primera página es la página **1**) y el número de asignaturas a devolver en cada página, respectivamente. También admite el parámetro `alumnos_gte` para devolver únicamente aquellas asignaturas que tienen ese número de alumnos o más. **Tened en cuenta que pueden aparecer los 3 parámetros a la vez, por lo que habrá que realizar el paginado a los resultados obtenidos del filtrado por número de alumnos**. Los valores de los tres parámetros deben ser números enteros y **cualquier otro parámetro se considera incorrecto**. Si los parámetros utilizados en la petición son incorrectos, se devolverá «**400 Bad Request**»; en otro caso se devolverá «**200 OK**» con el listado de asignaturas resultante. Ejemplos:

```
>>> muestra(requests.get(f"{URL}/asignaturas?page=1"))
400 BAD REQUEST
>>> muestra(requests.get(f"{URL}/asignaturas?page=2&per_page=3"))
200 OK
{
  "asignaturas": [
    "/asignaturas/3"
  ]
}
>>> muestra(requests.get(f"{URL}/asignaturas?alumnos_gte=50"))
200 OK
{
  "asignaturas": [
    "/asignaturas/0",
    "/asignaturas/3"
  ]
}
```

```
>>> muestra(requests.get(f"{URL}/asignaturas?alumnos_gte=50&page=1&per_page=1"))
200 OK
{
  "asignaturas": [
    "/asignaturas/0"
  ]
}
```

2.2. /asignaturas/X

Mediante esta ruta se podrá consultar una nueva asignatura, borrarla o modificarla:

- `DELETE`: borra una determinada asignatura. Si existe devuelve «**204 No Content**» y si no existe devuelve «**404 Not Found**». Ejemplo:

```
>>> muestra(requests.delete(f"{URL}/asignaturas/0"))
204 NO CONTENT
>>> muestra(requests.delete(f"{URL}/asignaturas/9999"))
404 NOT FOUND
```

- `GET`: Devuelve los datos de una asignatura concreta con código «**200 OK**» si existe, o «**404 Not Found**» si no existe. Ejemplo:

```
>>> muestra(requests.get(f"{URL}/asignaturas/1"))
200 OK
{
  "horario": [
    {
      "dia": "martes",
      "hora_final": 16,
      "hora_inicio": 14
    }
  ],
  "id": 1,
  "nombre": "GIW",
  "numero_alumnos": 40
}
>>> muestra(requests.get(f"{URL}/asignaturas/9999"))
404 NOT FOUND
```

- `PUT`: Reemplaza completamente una asignatura que existe usando los datos que se pasan en la petición, **pero conservando su identificador**. Si la asignatura no existe se devolverá «**404 Not Found**», y si existe se devolverá «**200 OK**». La asignatura que se pasa en la petición debe cumplir el esquema o se devolverá el código «**400 Bad Request**», que también se devolverá si la asignatura existe pero la petición **no tiene cuerpo**. Si se devuelve un error, **la asignatura no se modificará**. Ejemplos:

```
>>> muestra(requests.put(f"{URL}/asignaturas/9999", json={
...     "nombre": "GIW", "numero_alumnos": 40,
...     "horario": [{"dia": "martes", "hora_inicio": 14, "hora_final": 16}]}))
404 NOT FOUND
>>> muestra(requests.put(f"{URL}/asignaturas/1", json={
...     "nombre": "ALP", "numero_alumnos": 22, "horario": []}))
200 OK
```

```
>>> muestra(requests.put(f"{URL}/asignaturas/1", json={
...    "nombre": "ALP", "numero_alumnos": 22}))
400 BAD REQUEST
>>> muestra(requests.get(f"{URL}/asignaturas/1"))
200 OK
{
  "horario": [],
  "id": 1,
  "nombre": "ALP",
  "numero_alumnos": 22
}
```

- `PATCH`: Actualiza exactamente un campo de una asignatura que existe. Por lo tanto, en la petición se debe pasar un JSON con **un único campo** que deber ser **válido** según el esquema de las asignaturas. Si la asignatura no existe se devolverá «**404 Not Found**», y si existe se devolverá «**200 OK**». Si la asignatura existe pero el JSON pasado no es válido o la petición **no tiene cuerpo**, se devolverá «**400 Bad Request**». Si se devuelve un error, **la asignatura no se modificará**. Ejemplos:

```
>>> muestra(requests.patch(f"{URL}/asignaturas/1", json={"numero_alumnos": 1000}))
200 OK
>>> muestra(requests.patch(f"{URL}/asignaturas/1", json={"numero_alumnos": "tres"}))
400 BAD REQUEST
>>> muestra(requests.get(f"{URL}/asignaturas/1"))
200 OK
{
  "horario": [],
  "id": 1,
  "nombre": "ALP",
  "numero_alumnos": 1000
}
```

2.3. `/asignaturas/X/horario`

En esta ruta se puede obtener directamente el horario de una asignatura concreta.

- `GET`: Devuelve un JSON con un único campo `horario` con el horario de la asignatura en cuestión. Si esta asignatura existe se devolverá el código «**200 OK**» y si no «**404 Not Found**». Ejemplos:

```
>>> muestra(requests.get(f"{URL}/asignaturas/9999/horario"))
404 NOT FOUND
>>> muestra(requests.get(f"{URL}/asignaturas/3/horario"))
200 OK
{
  "horario": [
    {
      "dia": "jueves",
      "hora_final": 12,
      "hora_inicio": 10
    },
    {
      "dia": "viernes",
      "hora_final": 12,
      "hora_inicio": 10
    }
```

```
  ]
}
```

3. Tests de unidad

Manejar correctamente las distintas rutas implica tener en cuenta muchos matices, entre los que destaca detectar que la petición recibida es correcta y decidir el código de estado a devolver. Para que podáis comprobar con facilidad que vuestra API cumple con los requerimientos, he creado una serie de tests de unidad que realizan peticiones a la API en `http://127.0.0.1:5000` y comprueban que los resultados son los esperados. Todos los tests están incluidos en el fichero `tests.py` que os podéis descargar del Campus Virtual.

Los tests de unidad están realizados con la biblioteca estándar `unittest`, así que no tendréis que instalar nada adicional para poder lanzarlos. Para ejecutarlos debéis abrir un terminal en el directorio donde esté el fichero `tests.py` y escribir lo siguiente:

- Para lanzar **todos** los tests que existen:

```
$ python -m unittest tests.TestREST
```

- Para lanzar **un test concreto**, por ejemplo `test_get_horario_inexistente()`:

```
$ python -m unittest tests.TestREST.test_get_horario_inexistente
```

Importante: los test de unidad suponen que vuestra API REST se está ejecutando y escuchando en `http://127.0.0.1:5000`, que es el valor por defecto en Flask. En total, el tiempo necesario para ejecutar todos los tests de unidad no debería exceder de 5-10 segundos.